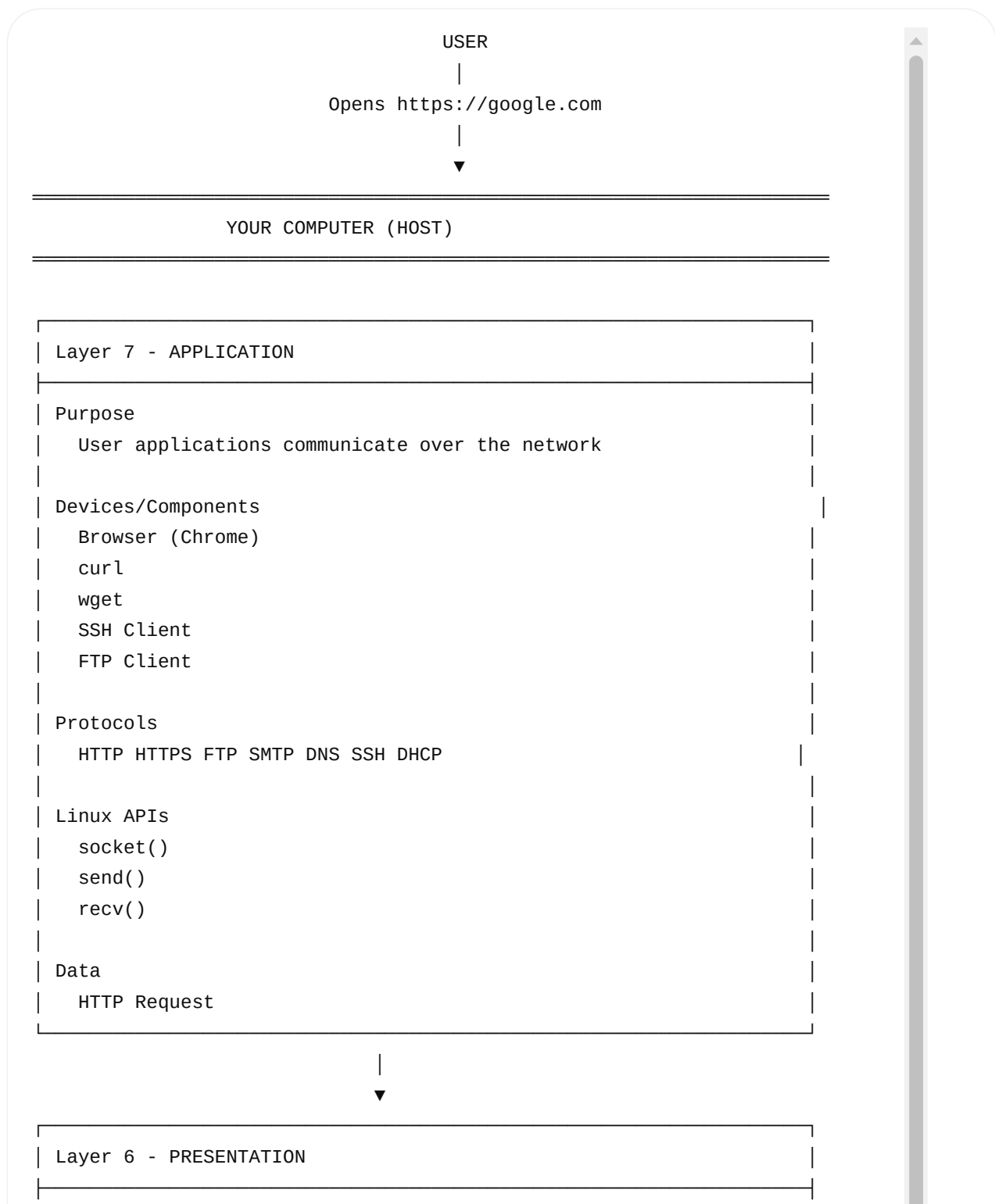


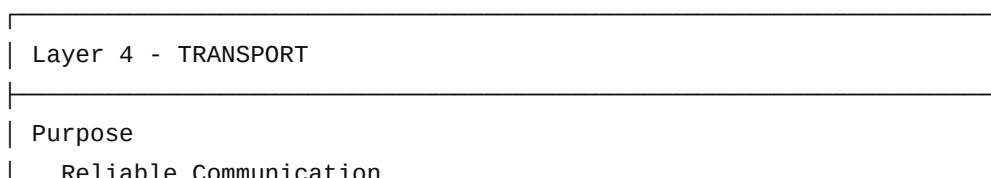
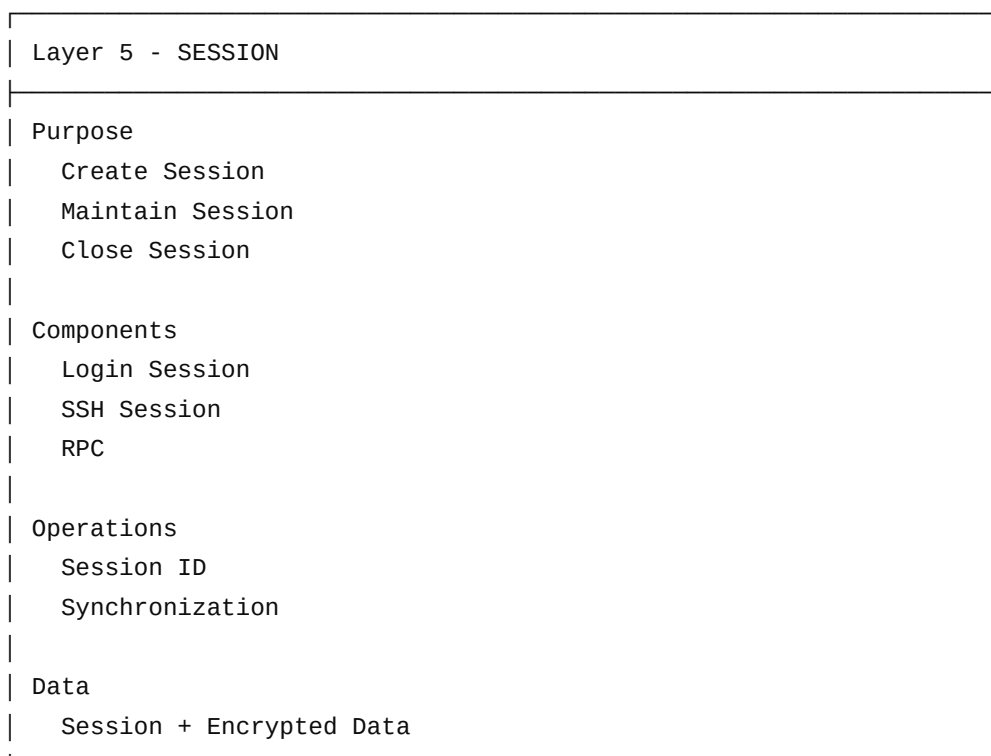
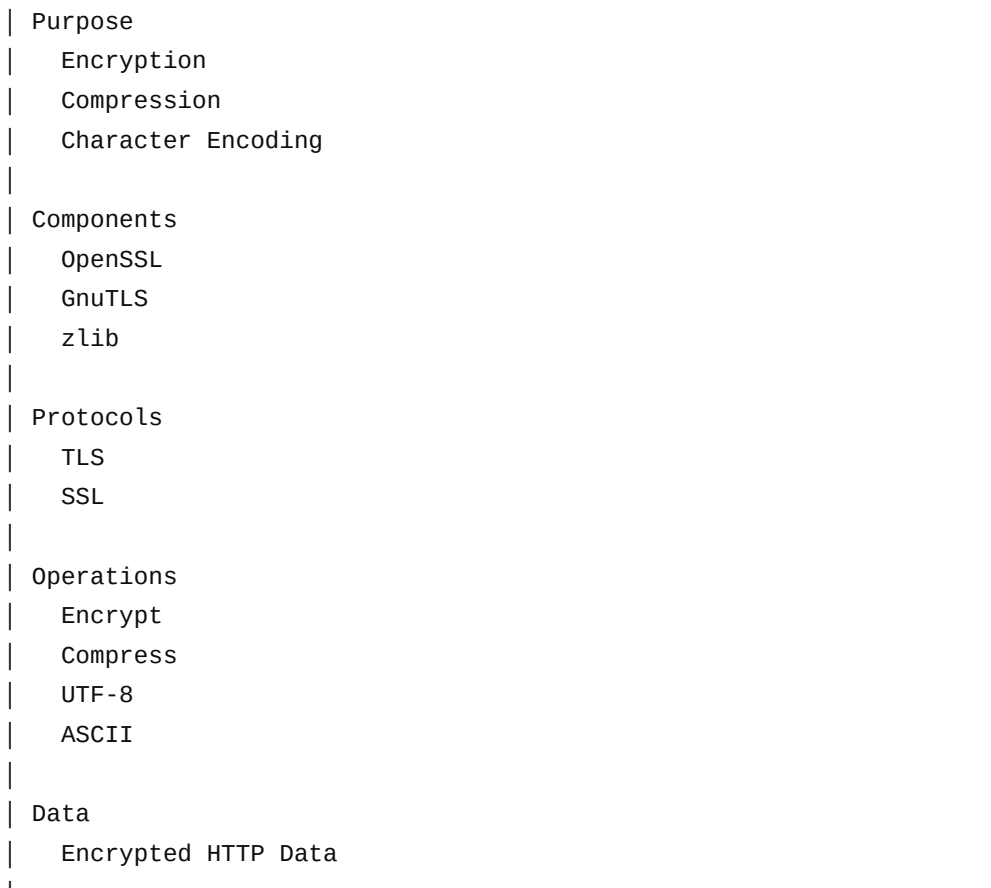
/visualize diagram. seven layers with used components
are devices

Excellent. This is one of the best ways to remember the OSI model for interviews.

Instead of memorizing the 7 layers, **associate each layer with the devices, software, protocols, and data it uses.**

OSI Model - Complete Visual Architecture





Segmentation
Flow Control
Protocols
TCP
UDP
Header
Source Port
Destination Port
Sequence Number
ACK Number
Flags
Linux Kernel
TCP Stack
UDP Stack
Data
TCP Segment



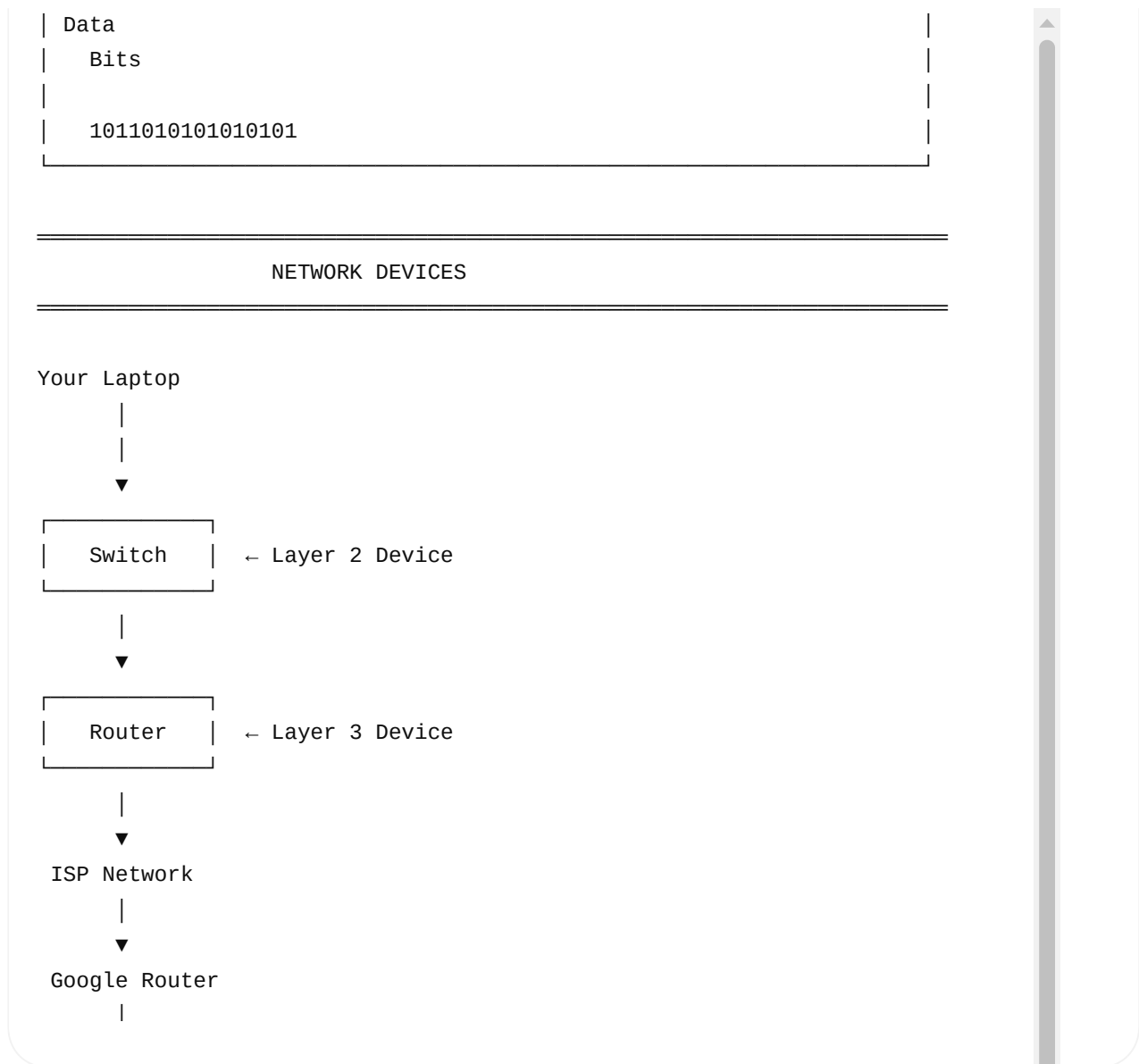
Layer 3 - NETWORK
Purpose
Routing
Logical Addressing
Devices
Router
Layer-3 Switch
Protocols
IPv4
IPv6
ICMP
Header
Source IP
Destination IP
TTL
Linux Kernel
IP Stack
Routing Table
Data
IP Packet

▼

Layer 2 - DATA LINK
Purpose
Local Network Delivery
Devices
Switch
Bridge
Wireless Access Point
Protocols
Ethernet
ARP
VLAN
Header
Source MAC
Destination MAC
EtherType
Trailer
CRC
Linux
NIC Driver
Data
Ethernet Frame

↓

Layer 1 - PHYSICAL
Purpose
Transfer Bits
Devices
Hub
Repeater
RJ45 Cable
Fiber
Wi-Fi Radio
NIC Hardware
Signal
Electrical
Optical
Radio



Which Device Works at Which Layer?

Device	OSI Layer	Purpose
Application (Chrome, Firefox, curl)	Layer 7	Create application requests
OpenSSL / GnuTLS	Layer 6	Encryption and decryption
SSH Session / RPC	Layer 5	Session management
TCP/UDP Stack (Kernel)	Layer 4	End-to-end transport
Router	Layer 3	Route packets using IP addresses
Layer-3 Switch	Layer 3	Switching with routing capability

Device	OSI Layer	Purpose
Switch	Layer 2	Forward frames using MAC addresses
Bridge	Layer 2	Connect LAN segments
Wireless Access Point	Layer 2	Bridge wireless and wired networks
NIC (Network Interface Card)	Layers 1 & 2	Frame handling and physical transmission
Hub	Layer 1	Repeat electrical signals
Repeater	Layer 1	Regenerate signals
Cable / Fiber / Radio	Layer 1	Physical transmission medium

Interview Memory Trick

Layer 7 → Applications (Chrome, SSH, FTP)

Layer 6 → Encryption (TLS/OpenSSL)

Layer 5 → Session (Login, SSH Session)

Layer 4 → TCP / UDP

Layer 3 → Router / IP

Layer 2 → Switch / MAC

Layer 1 → Cable / Fiber / Hub / Radio

End-to-End Visualization

User

|



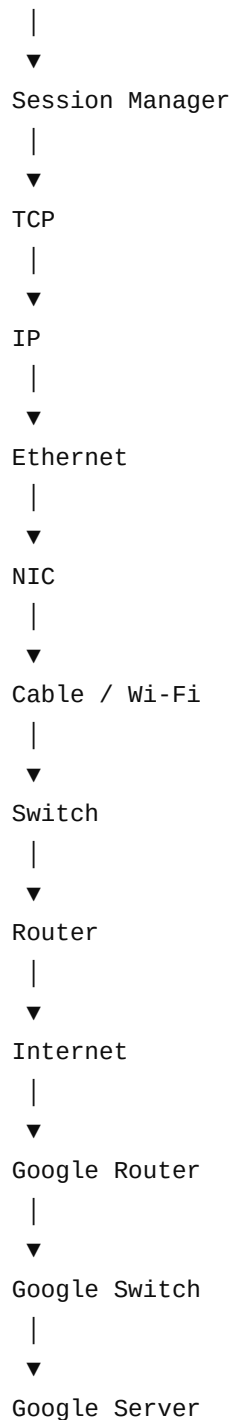
Chrome (Application)

|



OpenSSL (Presentation)





Embedded Linux Interview Tip

A common interview question is:

"When your C program calls `send()` , which OSI layers are involved?"

A good answer is:

- **Layer 7:** Your application prepares the data.
- **Layer 6:** If using HTTPS/TLS, a library such as OpenSSL encrypts the data.
- **Layer 5:** The communication session is maintained (conceptually).
- **Layer 4:** The Linux kernel adds the TCP or UDP header.
- **Layer 3:** The kernel adds the IP header and determines the route.

- **Layer 2:** The NIC driver builds the Ethernet frame using MAC addresses.
- **Layer 1:** The NIC hardware converts the frame into electrical, optical, or radio signals and transmits it.

This explanation demonstrates both networking knowledge and an understanding of how the Linux networking stack processes data.

/teach me with visualized clarify the below points with details

Transport Layer (Layer 4): The code that executes the TCP handshake, handles packet retransmissions, tracks port numbers, and splits data into Segments or Datagrams lives entirely in the kernel.
Network Layer (Layer 3): The kernel manages your IP Routing Tables and wraps data with IP Headers to create Packets. It handles global addressing and decides which network interface card (NIC) should receive the traffic.
Data Link Layer - Top Half (Layer 2): The kernel's network device drivers construct the Ethernet Frames and manage MAC addresses via ARP lookup tables.

Excellent question. This is exactly the level of understanding expected in **Embedded Linux, Linux Platform, Automotive, and Networking interviews**.

The biggest confusion for beginners is:

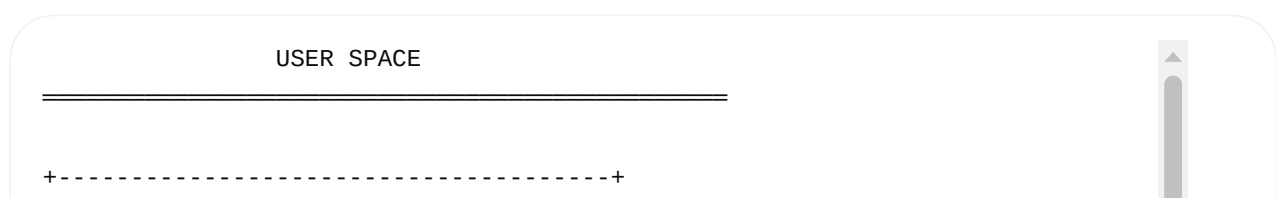
"If my application only calls `send()` , who performs all the networking work?"

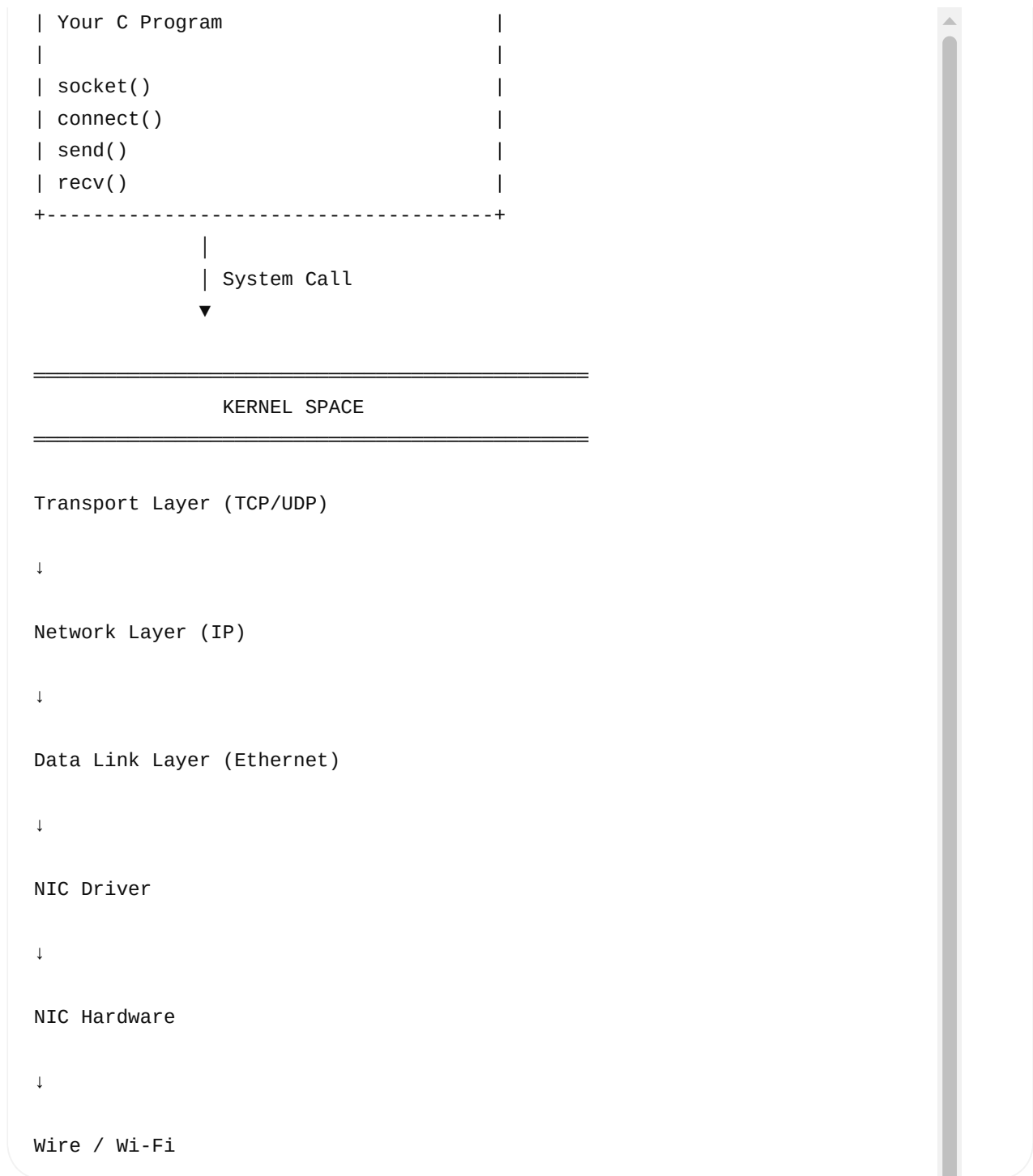
The answer is:

The Linux Kernel Networking Stack.

Let's visualize it from the moment your C program calls `send()` .

Big Picture





Notice:

Your application **stops after calling** `send()` .

Everything below is done automatically by the kernel.

Imagine Sending a Letter

Suppose you write

Hello Google

You simply hand it to the courier.

Do you

- build the truck?
- drive the truck?
- decide the road?
- decide which highway?

No.

Courier company does.

Linux Kernel is the courier company.

Step 1

Your Program

```
send(socket_fd,  
      buffer,  
      length,  
      0);
```

What happens?

Your program enters kernel mode.

Application

↓

send()

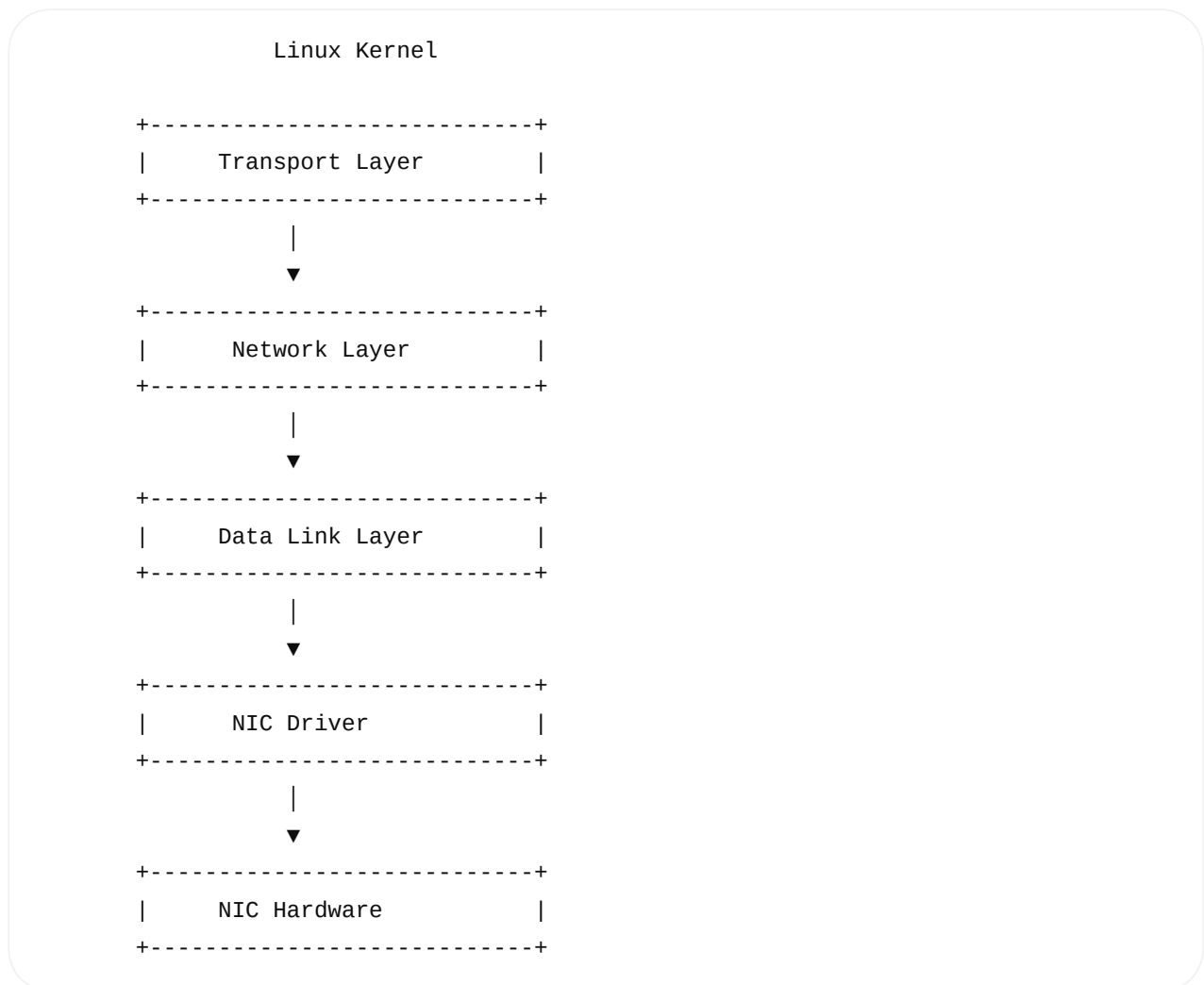
↓

Kernel

From this point,

Kernel becomes responsible.

Kernel Networking Stack



Let's study every layer.

Transport Layer (Layer 4)

Question

What does Transport Layer actually do?

Suppose

Application wants to send

Hello Google

Transport Layer asks

"How can I deliver this reliably?"

Responsibilities

Application Data

↓

TCP

↓

Reliable Delivery

↓

Port Numbers

↓

Segmentation

↓

Flow Control

↓

Acknowledgement

↓

Retransmission

Visualization

Application

Hello Google

↓

Transport Layer

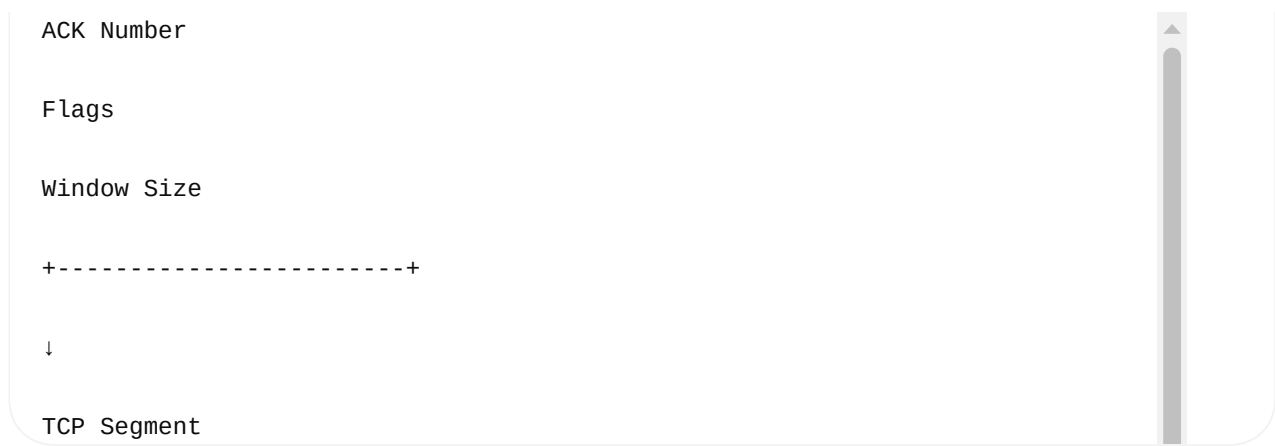
+-----+

Source Port

Destination Port

Sequence Number





Notice

Transport Layer creates

TCP Header

Where Does This Code Live?

Inside Linux Kernel

NOT

Inside your program.

Your application never executes

Three-way Handshake

Retransmission

Sliding Window

ACK Processing

Kernel executes all of it.

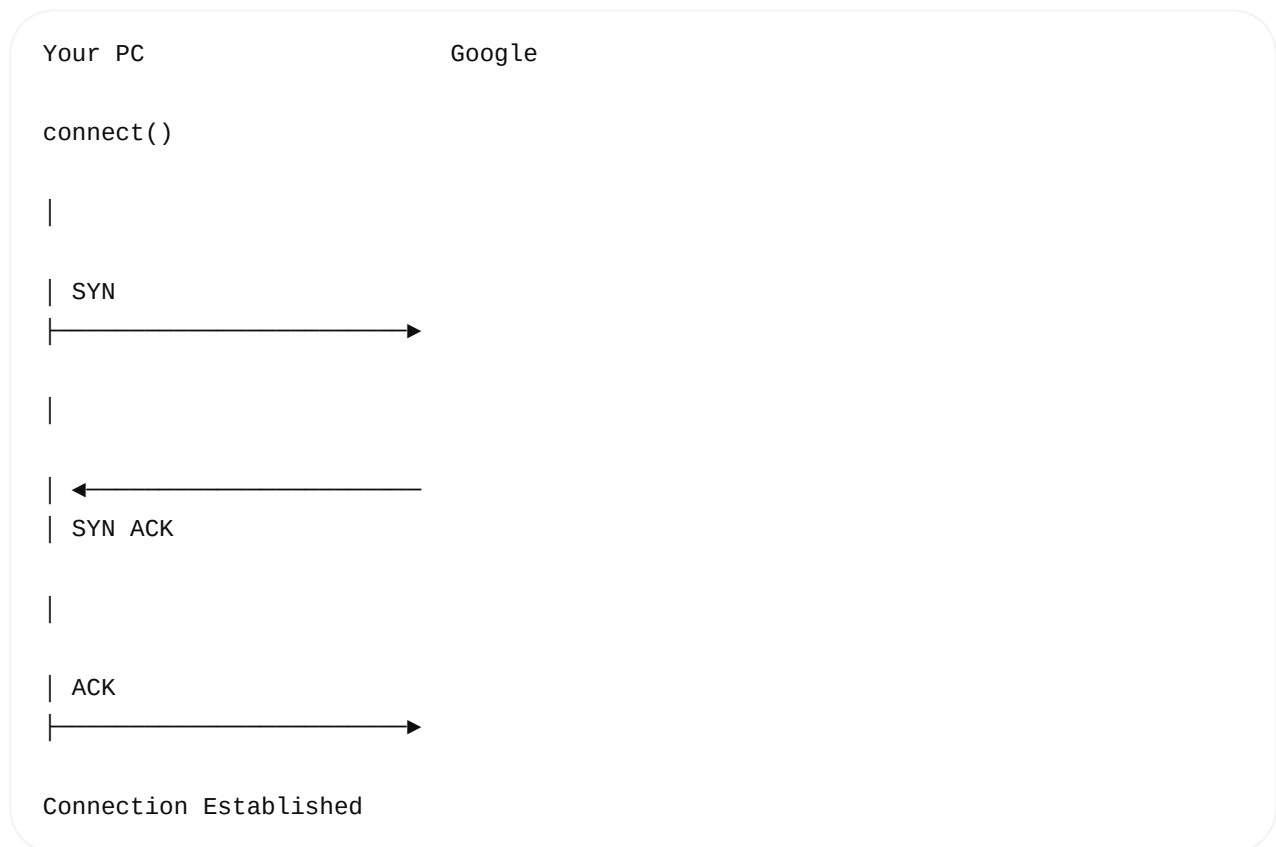
TCP Handshake Visualization

Your code

```
connect(sockfd, ...);
```

After this

Kernel starts



Question

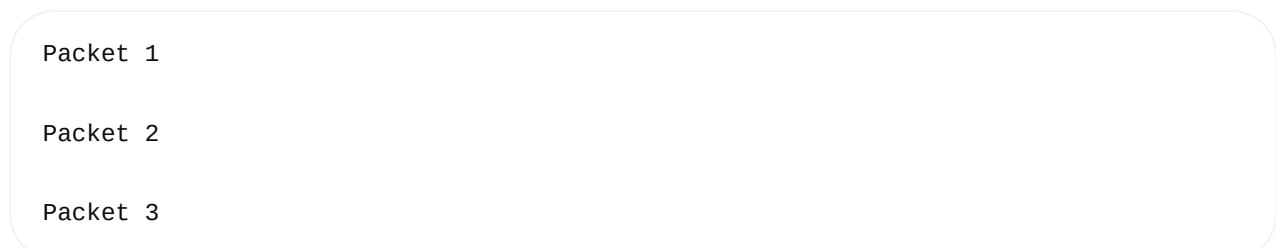
Who sent SYN?

Not your application.

Kernel did.

If Packet is Lost

Suppose



Packet 2 disappears.

Kernel notices

No ACK received.

Kernel automatically

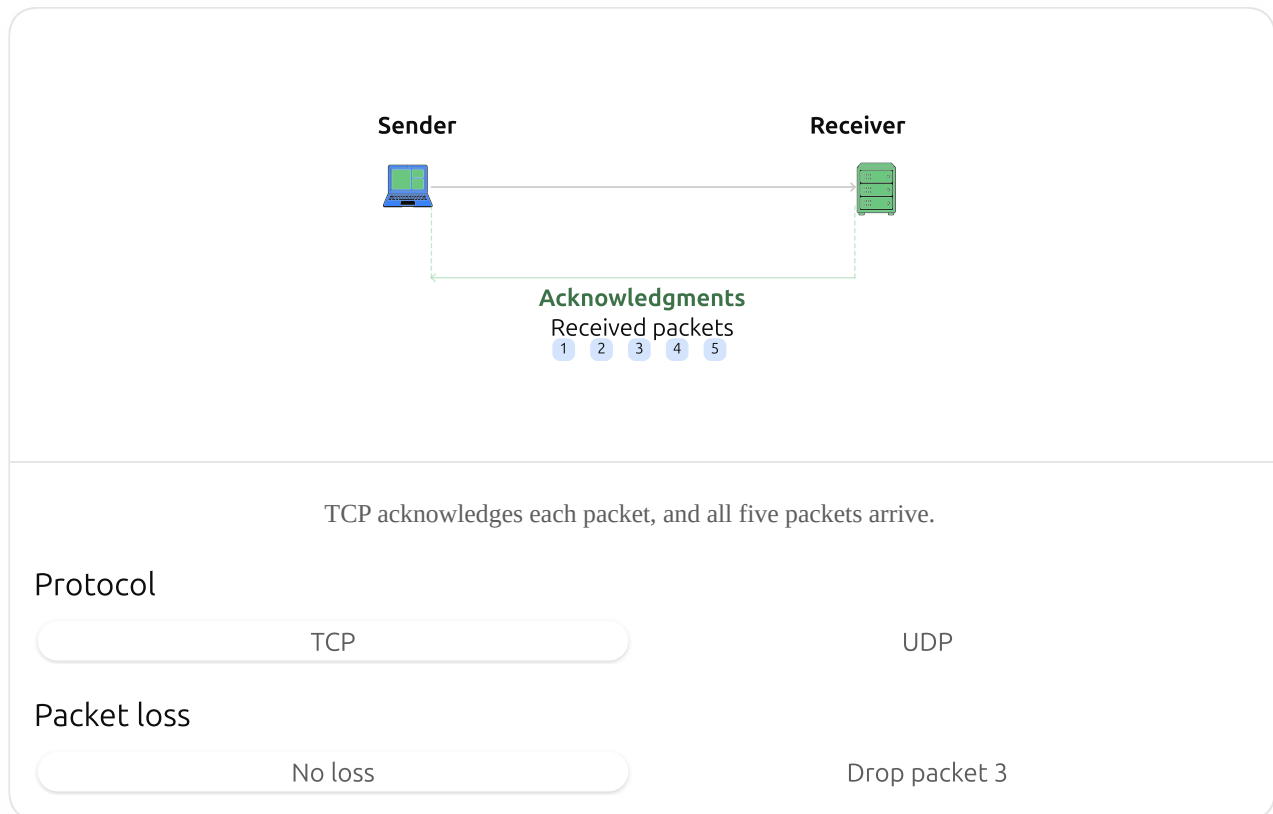
Packet 2

↓

Retransmit

Application knows nothing.

Visualization



This is why TCP is called **reliable**.

Port Numbers

Suppose

Chrome

SSH

FTP

all use networking.

Kernel identifies applications using ports.

Example

Chrome

↓

Source Port

52000

↓

Destination Port

443

Kernel inserts these ports into the TCP header.

Segmentation

Suppose

Application sends

10 MB

Ethernet cannot send

10 MB in one frame.

Kernel divides it.

Visualization

Application

10 MB

↓

Kernel

↓

Segment 1

Segment 2

Segment 3

Segment 4

↓

Network Layer

Application still thinks

"I sent 10 MB"

Kernel actually sent

hundreds or thousands of TCP segments.

Transport Layer Summary

Kernel performs

- ✓ TCP Handshake
- ✓ ACK
- ✓ Retransmission
- ✓ Segmentation
- ✓ Flow Control
- ✓ Port Management

Network Layer (Layer 3)

Transport finished.

Now kernel asks

"Where should this packet go?"

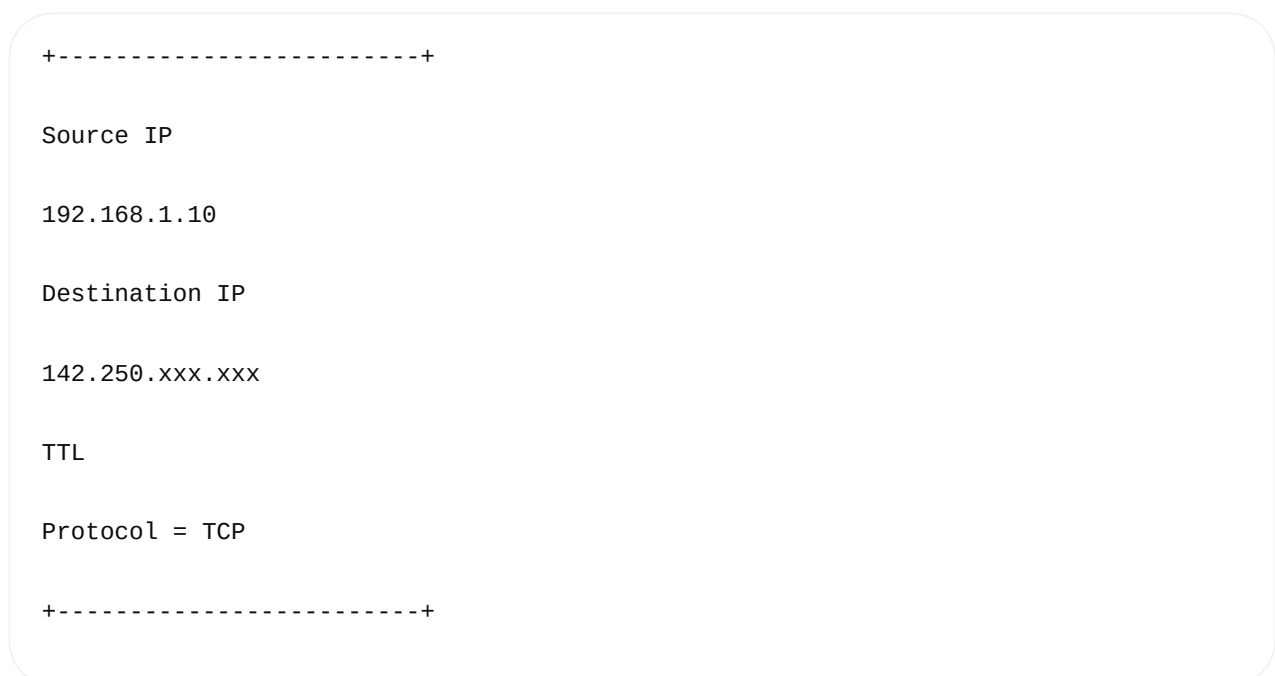
Visualization

TCP Segment



IP Header

Kernel creates



Now

TCP Segment becomes

IP Packet.

Where Does Destination IP Come From?

Application used



Kernel (with DNS resolution performed earlier by user-space resolver and DNS queries) now has the destination IP.

Example

```
www.google.com
```

↓

```
142.250.xxx.xxx
```

Routing Decision

Kernel now checks

Routing Table

Visualization

```
Routing Table
```

```
Destination
```

```
0.0.0.0
```

↓

```
Gateway
```

```
192.168.1.1
```

↓

```
Interface
```

```
eth0
```

Kernel asks

Which NIC should send this packet?

Example

Computer has

eth0

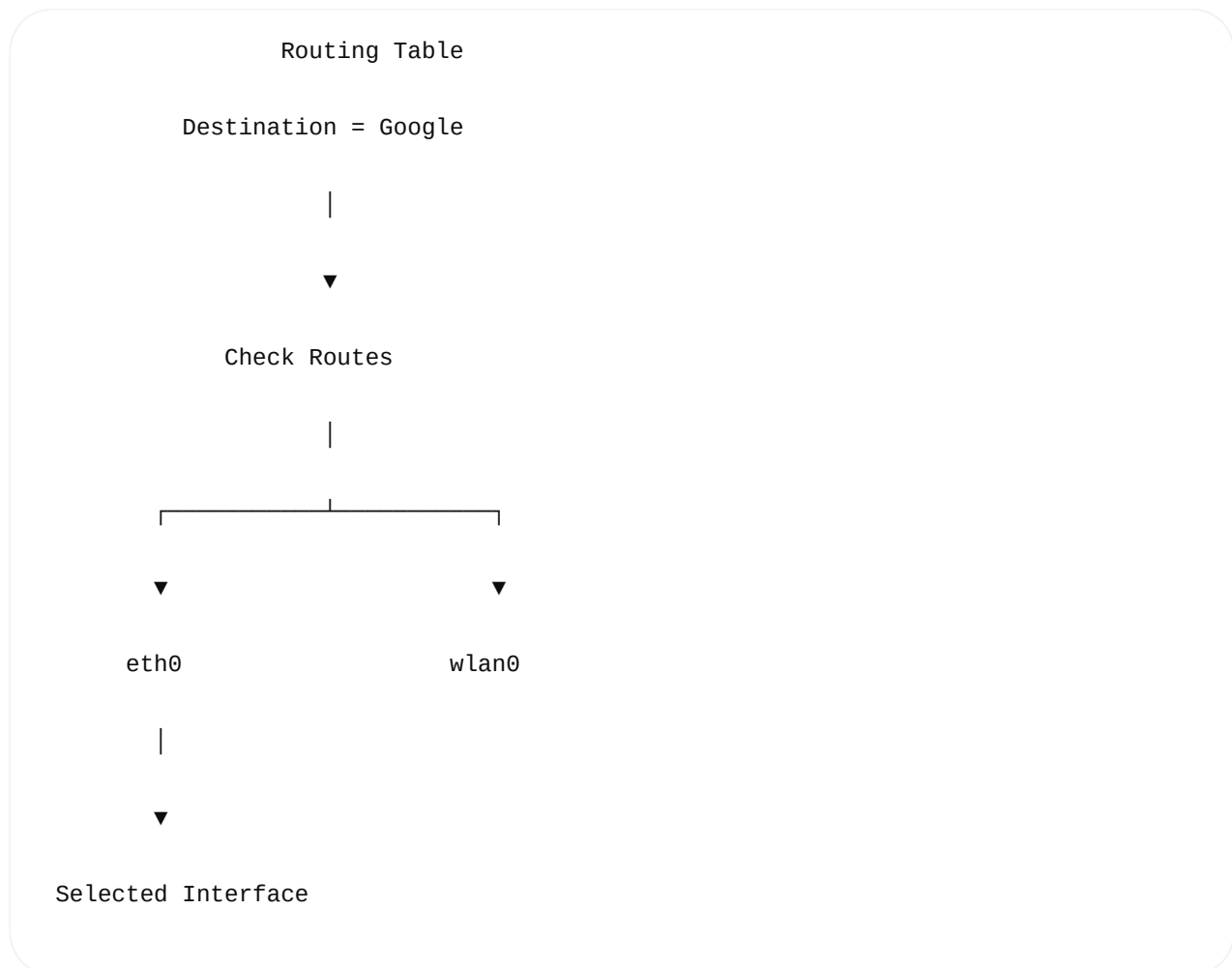
Wi-Fi

VPN

Kernel chooses

one interface.

Visualization



Application never chooses

Ethernet

or

Wi-Fi.

Kernel decides.

TTL

Kernel also inserts

TTL

Example

TTL = 64

Every router

64

↓

63

↓

62

↓

61

When

TTL

becomes

0

Packet is discarded.

This prevents routing loops.

Network Layer Summary

Kernel performs

- ✓ Routing
- ✓ IP Header Creation
- ✓ TTL

✓ Fragmentation (if required)

✓ Interface Selection

Data Link Layer (Layer 2)

Now packet reaches Ethernet.

Question

Can Ethernet use IP?

No.

Ethernet understands

MAC Address.

Visualization

IP Packet

↓

Ethernet

↓

Destination MAC ?

↓

ARP Lookup

ARP Table

Kernel checks

IP

↓

MAC

Example

192.168.1.1

↓

00:11:22:33:44:55

Kernel stores this in

ARP Cache.

If MAC Unknown

Kernel broadcasts

Who has

192.168.1.1 ?

Router replies

I am

192.168.1.1

MAC

00:11:22:33:44:55

Kernel stores it.

Ethernet Frame Construction

Kernel creates

+-----+

Destination MAC

Source MAC

EtherType

IP Packet



CRC

+-----+

Notice

Transport created

TCP Header.

Network created

IP Header.

Data Link creates

Ethernet Header.

Complete Visualization

Application Data

↓

```
+-----+
| TCP Header          |
+-----+
```

↓

```
+-----+
| IP Header           |
| TCP Header          |
+-----+
```

↓

```
+-----+
| Ethernet Header     |
| IP Header           |
| TCP Header          |
| Data                |
| CRC Trailer         |
+-----+
```

This is called

Encapsulation.

NIC Driver

Kernel now passes

Ethernet Frame

to

NIC Driver.

Driver

knows

Intel

Realtek

Broadcom

hardware registers.

Driver tells hardware

Transmit this frame.

NIC Hardware

Finally

NIC converts

Frame

into

10110101010101010...

Electrical

Optical

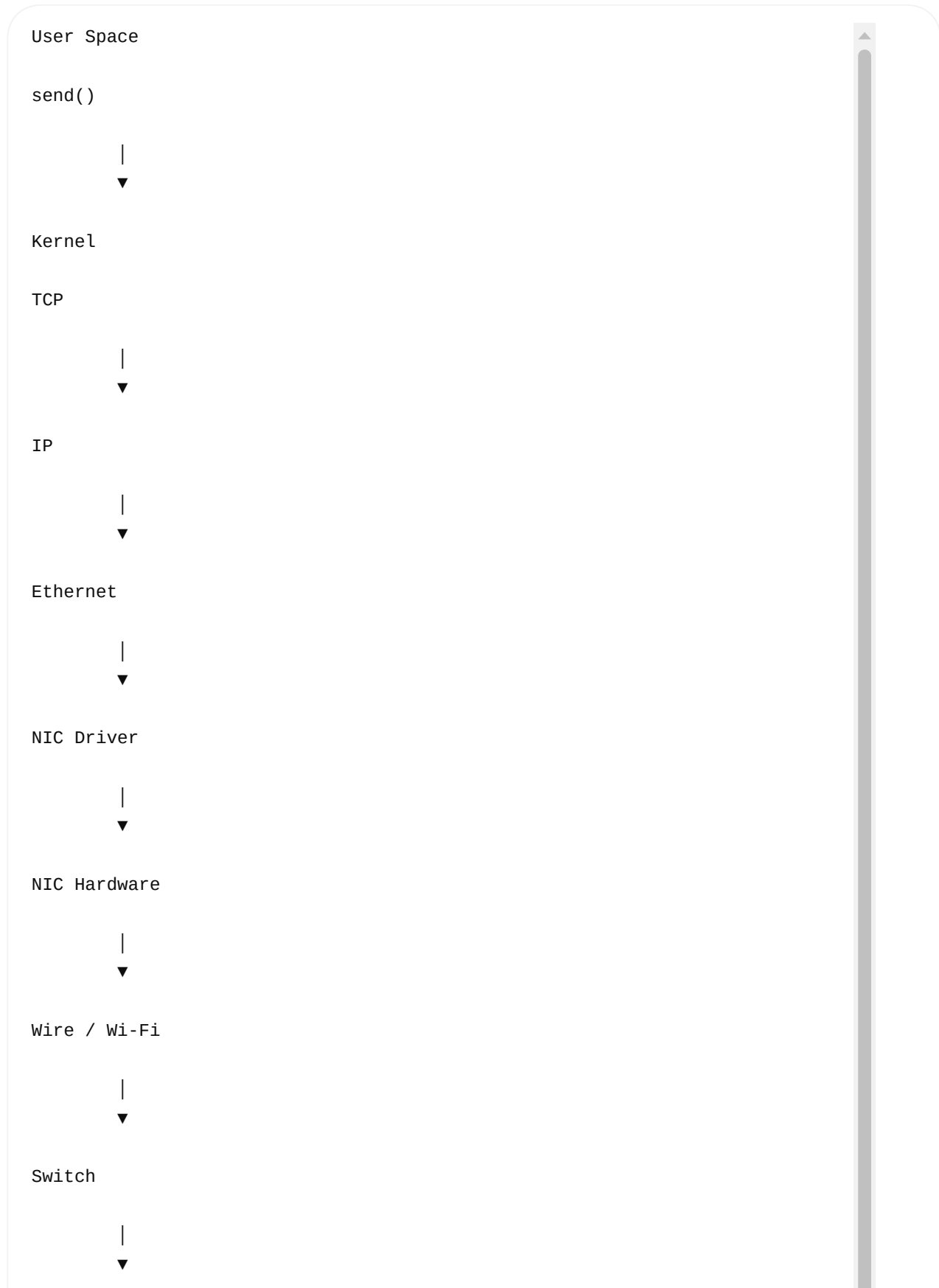
or

Radio

signals

and transmits it.

Complete Journey





Interview Summary

Layer	Kernel Responsibility	Header Added	Key Data
Transport (L4)	TCP handshake, segmentation, ACKs, retransmissions, flow control, port management	TCP/UDP Header	Segment/Datagram
Network (L3)	Routing, IP addressing, interface selection, TTL, fragmentation	IP Header	Packet
Data Link (L2)	ARP lookup, MAC addressing, Ethernet frame creation, CRC	Ethernet Header + Trailer	Frame
Physical (L1)	Driver programs the NIC; NIC hardware transmits bits over cable, fiber, or radio	None	Bits

Embedded Linux Interview Questions

- When does the kernel perform the TCP three-way handshake?**
 - During `connect()` for a client or while processing incoming connection requests for a listening server.
- Does `send()` transmit data directly to the network card?**
 - No. `send()` copies data into the kernel's networking stack. The kernel then processes TCP/UDP, IP, Ethernet, and finally hands the frame to the NIC driver.
- Who creates the TCP, IP, and Ethernet headers?**
 - The Linux kernel networking stack. Applications using standard sockets provide payload data; the kernel constructs the protocol headers automatically.
- Who decides whether to use `eth0`, `wlan0`, or another interface?**
 - The kernel consults the routing table and selects the appropriate outgoing network interface based on the destination IP and routing rules.

